CMSC436 Midterm Exam — Dedicated Study Guide

Exam Logistics: Thursday in Lecture (1h15m / 75 mins) • Scope: Lectures 0 through 5 (Touches up to Slide 49) & Toast • Format: Spring 2026 Midterm Style

1. Exam Blueprint, Scope Boundaries & Reference Sheet Strategy

Your Thursday midterm lasts **75 minutes** and mirrors the two-part structure of the posted Spring 2026 midterm (totaling ~97–100 points) accompanied by a printed 2-page API Documentation Sheet. Knowing exactly what is in scope—and what is already printed on the reference sheet—lets you focus your study time on the exact code patterns and conceptual traps graded on the exam.

Exam Component	Weight	What Is Tested & How to Prepare
Part A: Short Questions	~38 pts (15–18 Qs)	Multiple-choice and 1–2 line fill-in-the-box questions (1–4 pts each) covering XML attributes, resource syntax (`@color/c`, `@+id/exam`), styles (`30sp`), programmatic layout setup, `View.OnTouchListener` & `MotionEvent`, `finish()`, `Toast`, Activity stack & lifecycle order, and `SharedPreferences`.
Part B.1: The Model	~14 pts	Completing a pure Kotlin Model class (like `Hangman`, `TicTacToe`, or `Mortgage`): calling helper methods in a constructor, logging with `Log.w`, initializing a 1D or 2D generic `Array<T>` with a lambda, writing a `0..length - 1` loop, and declaring a `companion object` constant.
Part B.2: XML Layout	~4 pts	Adding attributes to a widget inside a `RelativeLayout`: assigning an ID (`android:id="@+id/grid"`) and positioning relative to another sibling (`android:layout_below="@id/button"`).
Part B.3: MainActivity	~41 pts	Completing the Controller: fetching screen width via `DisplayMetrics`, retrieving views with `findViewById<T>(R.id.x)`, configuring a `GridLayout`, instantiating and styling an `Array<TextView>` or `Array<Array<Button>>`, adding views with `grid.addView(v, w, h)`, implementing `View.OnClickListener`, reading `EditText` input, invoking Model methods, and disabling buttons on win/loss.

Strict Scope Check: Included vs. Excluded Topics

- **Included Slide Decks**: `0-Overview`, `1-HelloAndroid`, `2A-KotlinBasics`, `2B-TipCalculator` (+ `Bindings` & `WebView`), `3-TicTacToe`, `4-Mortgage`, `5-Touches` **up to Slide 49 inclusive**, and `Toast`.

- **Excluded from This Midterm**:
  - **Swipes, Taps & Gestures (`5-Touches` Slides 50–72)**: `GestureDetector`, `GestureDetector.OnGestureListener` (`onFling`, `onDown`, `onScroll`), and `GestureDetector.OnDoubleTapListener` (`onDoubleTap`, `onSingleTapConfirmed`) are **NOT** included. *(Skip Questions 8 and 9 on the Spring 2026 midterm key.)*
  - **2D Graphics Drawing (`Paint` & `Canvas`)**: Not covered in your pre-midterm lecture sequence. *(Skip Questions 16 and 17 on the Spring 2026 midterm key.)*

Documentation Sheet vs. What You Must Memorize

The exam provides a 2-page documentation sheet (identical in style to `midtermSpring2026Documentation.pdf`). It lists common XML attributes, resource syntax rules, and signatures for `View`, `TextView`, `ViewGroup.addView`, `View.OnClickListener`, `SharedPreferences`, `Toast`, `DisplayMetrics`, `GridLayout`, and the `Array` constructor. However, the sheet **does NOT list** the items below—you must know them from memory:

- **Touch Handling APIs (Slides 1–49 of `5-Touches`)**: The method name `onTouch(v: View?, event: MotionEvent?): Boolean`, the return value meaning (`true` = consumed, `false` = propagated), the class name `MotionEvent`, its constants (`ACTION_DOWN`, `ACTION_MOVE`, `ACTION_UP`), its coordinate methods (`rawX`, `rawY`, `x`, `y`, `action`), `findViewById(android.R.id.content)`, and moving a view via `RelativeLayout.LayoutParams(w, h)` with `leftMargin` and `topMargin`.
- **Activity Stack & Lifecycle**: The 7 lifecycle method names (`onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart`, `onDestroy`) and the exact order they execute on launch, backgrounding, return, and `finish()`.
- **Multi-Activity Navigation & Data Sharing**: `Intent(this, SecondActivity::class.java)`, `startActivity(intent)`, `finish()`, manifest `<activity>` registration, and `companion object` static sharing.
- **XML Specifics Not on the Sheet**: `android:inputType="numberDecimal"`, `<style name="nice"><item name="android:textSize">30sp</item></style>`, and `<RadioGroup>` wrapping `<RadioButton>` elements.

---

2. Part A High-Yield Rapid Reference (Syntax & Concept Traps)

A. XML Layouts, Resources, Styles & Widgets

- **Declaring a resource** (in `res/values/strings.xml` or `colors.xml`):

```
<string name="message">HI THERE</string>
<color name="c">#FF0000</color>
```

- **Referencing a resource in XML vs. Kotlin**:

    - In XML: `@type/name` (singular—no "s"!). Example: `android:background="@color/c"`, `android:text="@string/message"`, `style="@style/nice"`, `android:icon="@mipmap/hi"`.
    - In Kotlin: `R.type.name`. Example: `R.layout.activity_main`, `R.id.grid`, `R.anim.slide_from_left`.

- **Assigning vs. Referencing Component IDs in XML**:
    - Give a component an ID: `android:id="@+id/grid"`
    - Position relative to another component's ID in `RelativeLayout`: `android:layout_below="@id/button"` or `android:layout_above="@+id/exam"` (the documentation sheet notes adding `+` after `@` when referring to another element by ID, and both `@id/` and `@+id/` are accepted on the key).

- **Defining a Style in `styles.xml` / `themes.xml` (Spring 2026 Q5)**:

```
<style name="nice" parent="@style/TextAppearance.AppCompat">
    <item name="android:textSize">30sp</item>
    <item name="android:padding">10dp</item>
</style>
```

  Trap: Always include units—use `sp` for text sizes (`30sp`) and `dp` for dimensions/margins/padding (`10dp`).

- **Widget & Layout Essentials**:
    - **Restrict `EditText` to real/decimal numbers (Spring 2026 Q1)**: `android:inputType="numberDecimal"` (NOT `"real"` or `"float"`).
    - **Mutually exclusive radio buttons (Quiz 2 Q6)**: Place `<RadioButton>` elements inside a `<RadioGroup>` element. Check state in Kotlin with `rb10.isChecked`.
    - **Horizontal divider line**: `<View android:layout_width="match_parent" android:layout_height="5dp" android:background="#FF0000" />`.
    - **Margins vs. Padding**: **Margin** is space *outside* the view (`android:layout_marginTop="50dp"`); **Padding** is space *inside* the view around its text/content (`android:padding="10dp"`).
    - **Table-like layouts**: `TableLayout` (uses `<TableRow>` children, inherits from `LinearLayout`) or `GridLayout` (uses `rowCount` and `columnCount`, inherits directly from `ViewGroup`).

B. Multi-Activity Navigation, Stack & Lifecycle (Guaranteed Exam Questions)

Activities in an app are managed in a **Stack** (LIFO). The user sees the Activity at the top of the stack.

- **Start a new activity (pushes onto stack)**:

```
var myIntent : Intent = Intent( this, DataActivity::class.java )
this.startActivity( myIntent )
```

  And in `AndroidManifest.xml`, add an `<activity>` tag inside `<application>`: `<activity android:name=".DataActivity" />`.

- **End current activity and go back (pops off stack — Spring 2026 Q11)**:

```
fun goBack( ) : Unit {
    finish( ) // or this.finish( )
}
```

Stack Trace Example (Fall 2025 Quiz 2 Q1): App starts with `A` → starts `B` → starts `C` → starts `D` (stack bottom-to-top: `A, B, C, D`) → finishes `D` (top is now `C`) → finishes current activity: **we just finished `C`, and `B` is now executing!**

Scenario / Transition	Exact Callback Sequence	Crucial Exam Distinctions (Q13–Q15 & Quiz 2)
1. Activity First Created (pushed onto stack)	`onCreate` → `onStart` → `onResume`	`onCreate` runs **ONLY once** when pushed onto the stack (never when returning to the activity).
2. Activity Goes to Background (another activity starts on top, or Home pressed)	`onPause` (1st) → `onStop` (2nd)	`onPause` runs when losing foreground focus; `onStop` runs when invisible. `onDestroy` is **NOT** called!
3. Activity Returns to Top of Stack (prior activity on top finishes)	`onRestart` → `onStart` → `onResume`	**`onRestart`** is the ONLY method called when returning to the top of the stack that is **NOT** called after `onCreate` (Q15). **`onStart`** and **`onResume`** are called *both* after `onCreate` and when returning to the top of the stack (Q14). Call `updateView()` in `onStart()`!
4. Activity Ends / Popped Off Stack (`finish()` called)	`onPause` (1st) → `onStop` (2nd) → `onDestroy` (3rd)	Exact order tested in Spring 2026 Q13! **`onDestroy`** runs *only* when the activity ends, not when backgrounded.

C. Sharing Data, Persistent Storage (`SharedPreferences`) & `Toast`

- **Sharing an object across Activities via `companion object` (Quiz 2 Q7 & Spring 2026 B.1)**:

```
// Inside MainActivity (or Hangman):
companion object {
    lateinit var mortgage : Mortgage
    val GUESSES : Int = 5
}
// Accessing from DataActivity (another class):
var m : Mortgage = MainActivity.mortgage
```

- **Persistent Local Storage with `SharedPreferences` (Spring 2026 Q18 & Quiz 2)**:

- Data structure: Organized as a **list/set of key/value pairs** (like a hashtable).
- Obtain `SharedPreferences` (inside an Activity or using a `Context` reference):

```
var sp : SharedPreferences = getSharedPreferences( "XYZ", Context.MODE_PRIVATE )
// In Mortgage slides: context.getSharedPreferences( context.packageName +
"_preferences", Context.MODE_PRIVATE )
```

- Write to preferences (requires `SharedPreferences.Editor` + `commit()`):

```
var editor : SharedPreferences.Editor = sp.edit( )
editor.putInt( "a", 3 )
editor.putFloat( "rate", 0.035f )
editor.commit( ) // Mandatory to save changes!
```

- Read from preferences (second argument is the default value if key is not found on first run):

```
var a : Int = sp.getInt( "a", 1 )
var r : Float = sp.getFloat( "rate", 0.035f )
```

- **Building and Showing a `Toast` (Spring 2026 Q12)**:

```
var t : Toast = Toast.makeText( this, "HI", Toast.LENGTH_LONG ) // or
Toast.LENGTH_SHORT
t.show( )
```

D. Touch Events & Moving a View (`5-Touches` Slides 1–49 — High Priority!)

Because Swipes and Gestures (Slides 50+) are excluded, all touch questions on your exam come directly from **Slides 1–49 of `5-Touches.pptx`**:

- **Interface & Single Callback Method (Spring 2026 Q7)**:
  - Interface: `View.OnTouchListener` (a public static inner interface of `View`).
  - Only method: `override fun onTouch( v : View?, event : MotionEvent? ) : Boolean`
  - Return value: Return `true` if the event is **consumed**; return `false` if the event is **propagated** to Views underneath.
  - Registering on a View v: `v.setOnTouchListener( th )` (or `v.setOnTouchListener( this )` if `MainActivity` implements `View.OnTouchListener`).
  - Getting the top-level View containing the entire screen: `var view : View = findViewById( android.R.id.content )`.
- **The `MotionEvent` Class (Spring 2026 Q10)**:
  - Action constants inside `MotionEvent`: `MotionEvent.ACTION_DOWN` (finger touched screen), `MotionEvent.ACTION_MOVE` (finger moving across screen), `MotionEvent.ACTION_UP` (finger lifted off screen).
  - Retrieving the action in Kotlin: `var action : Int = event.action` (or `event.getAction()`; test `if (event != null)` first).

- Touch sequence when swiping/dragging on screen: **One `ACTION_DOWN` → Many discrete `ACTION_MOVE` events → One `ACTION_UP`**.
- Coordinate methods (all return `Float`):
  - `event.rawX` / `event.getRawX()` and `event.rawY` / `event.getRawY()`: X and Y coordinates within the **entire screen**.
  - `event.x` / `event.getX()` and `event.y` / `event.getY()`: X and Y coordinates within the **View `v` that originated the event**.
  - *Note*: When the touch listener is registered on `android.R.id.content`, `rawX == x` and `rawY == y`.

- **Moving/Dragging a Label (`TextView`) with a Finger (Slides 25–49)**:

```
// 1. Inside onCreate: place tv inside a RelativeLayout using
RelativeLayout.LayoutParams(w, h)
var rl : RelativeLayout = RelativeLayout( this )
params = RelativeLayout.LayoutParams( 300, 200 ) // width = 300, height = 200
params.leftMargin = 50   // starting X coordinate of tv
params.topMargin = 150   // starting Y coordinate of tv
tv = TextView( this )
tv.setBackgroundColor( Color.BLUE )
rl.addView( tv, params )
setContentView( rl )
tv.setOnTouchListener( this )

// 2. Inside onTouch( v : View?, event : MotionEvent? ) : Boolean
if ( event != null ) {
    var action : Int = event.action
    if ( action == MotionEvent.ACTION_DOWN ) {
        startX = params.leftMargin
        startY = params.topMargin
        startTouchX = event.rawX.toInt( )
        startTouchY = event.rawY.toInt( )
    } else if ( action == MotionEvent.ACTION_MOVE ) {
        params.leftMargin = startX + ( event.rawX.toInt( ) - startTouchX )
        params.topMargin = startY + ( event.rawY.toInt( ) - startTouchY )
        tv.layoutParams = params // or tv.setLayoutParams( params )
    }
}
return true
```

---

3. Part B Master Code Patterns (Model, XML & Controller)

Part B (worth ~60% of the exam) asks you to fill in 12–18 code boxes across a Model class, an XML layout, and `MainActivity`. Memorize these exact idioms from lecture and the Spring 2026 key:

Pattern 1: 1D & 2D Generic Array Instantiation with Lambdas

```
// 1D Array of Booleans of length word.length, all initialized to false (Spring 2026 B.1):
indexes = Array<Boolean>( word.length, { i -> false } )

// 1D Array of 5 TextViews instantiated in MainActivity (Spring 2026 B.3):
tvs = Array<TextView>( 5, { i -> TextView( this ) } )

// 2D Array of Buttons (side x side) in TicTacToe:
buttons = Array<Array<Button>>( side, { i -> Array<Button>( side, { j -> Button( this ) }
) } )
```

Pattern 2: Screen Width, `GridLayout` & Adding Views in `onCreate`

```
// 1. Retrieve DisplayMetrics and screen width in pixels (Spring 2026 B.3):
var dm : DisplayMetrics = resources.displayMetrics // or
Resources.getSystem().displayMetrics
var width : Int = dm.widthPixels

// 2. Instantiate GridLayout by code OR retrieve from XML:
var g : GridLayout = GridLayout( this )            // by code (Spring 2026 Q6)
grid = findViewById<GridLayout>( R.id.grid )       // from XML (Spring 2026 B.3)
grid.rowCount = 1                                  // or grid.columnCount = side

// 3. Configure each View in an array and add to GridLayout with (width, height):
for ( i in 0..tvs.size - 1 ) {
    tvs[i].text = ""
    tvs[i].setBackgroundColor( Color.YELLOW )
    grid.addView( tvs[i], width / 6, width / 6 )
}
```

Spanning multiple cells in a `GridLayout` (TicTacToe status bar):

```
var rowSpec : GridLayout.Spec = GridLayout.spec( TicTacToe.SIDE, 1 )    // start row, row
span
var columnSpec : GridLayout.Spec = GridLayout.spec( 0, TicTacToe.SIDE ) // start col, col
span
var lp : GridLayout.LayoutParams = GridLayout.LayoutParams( rowSpec, columnSpec )
status.layoutParams = lp
grid.addView( status )
```

Pattern 3: Event Listeners, Lambdas vs. Inner Classes & Dialogs

- **When can you use a Lambda expression? (Fall 2025 Quiz 2 Q4)**: **ONLY when the listener interface has exactly 1 abstract method** (a functional interface, such as `View.OnClickListener`, `View.OnTouchListener`, or `DialogInterface.OnClickListener`).

  ```
  buttons[i][j].setOnClickListener { update( i, j ) }
  ```

- **When MUST you implement all methods in a class? (Spring 2025 Quiz 2 Q4)**: When implementing an interface with multiple methods like `TextWatcher` (which has **3 abstract methods**: `beforeTextChanged`,

`onTextChanged`, `afterTextChanged`), you must implement **all 3 methods**, leaving unused ones empty `{}`, and register with `editText.addTextChangedListener(tch)`.

- **Inner Class `ButtonHandler` + Reading `EditText` + Updating GUI (Spring 2026 B.3)**:

```
inner class ButtonHandler : View.OnClickListener {
    override fun onClick( p0 : View? ) {
        var s : String = playET.text.toString( )
        if ( s != null && s.length > 0 ) {
            h.play( s[0] )                              // pass first Char to Model
            leftTV.text = "" + h.getGuessesLeft( )      // convert Int to String with
"" + ...
            updateArray( )
            if ( h.gameOver( ) == 1 ) {
                button.isEnabled = false                // disable button on win
                grid.setBackgroundColor( Color.GREEN )  // set GridLayout background
            }
        }
    }
}
// Inside onCreate:
var bh : ButtonHandler = ButtonHandler( )
button.setOnClickListener( bh )
```

- **`AlertDialog.Builder` and Accessing Outer Activity (`this@MainActivity`)**:

```
inner class PlayDialog : DialogInterface.OnClickListener {
    override fun onClick( dialog : DialogInterface?, id : Int ) {
        if ( id == -1 ) {        // -1 = Positive ("YES") button
            ttt.resetGame( )
        } else if ( id == -2 ) { // -2 = Negative ("NO") button
            this@MainActivity.finish( )
        }
    }
}
```

---

4. Targeted Practice Exam Questions (Adjusted for This Thursday's Scope)

Use these practice questions (combining the high-yield Spring 2026 midterm items with new replacement questions for `5-Touches` Slides 1–49, `4-Mortgage`, and Quiz 2) to test yourself under timed conditions. Check your work against the Answer Key below.

Part A Practice: Short Answer & Code Completion

1. Inside an `EditText`, what XML attribute and value constrain the user to only enter real/decimal numbers (like `3.5`)?
2. Inside a `Button` element in a `RelativeLayout`, write the XML attribute that positions the `Button` to the right of the View whose id is `label_bill`.

3. Inside `styles.xml`, write the complete XML element defining a style named `headerStyle` whose text size is `28sp`.
4. What is the name of the only method in `View.OnTouchListener`, what are its two parameter types, and what is its return type?
5. Inside `onTouch`, what is the difference between returning `true` and returning `false`?
6. In the `MotionEvent` class, what is the difference between `event.rawX` and `event.x`? Under what setup are they equal?
7. When a user touches the screen, slides their finger across the screen, and lifts their finger up, how many `ACTION_DOWN`, `ACTION_MOVE`, and `ACTION_UP` events occur?
8. Inside `MainActivity`, write one line of Kotlin code to retrieve the top-level `View` containing everything in the current screen and assign it to `var topView : View`.
9. In an app with 3 activities (`MainActivity`, `SecondActivity`, `ThirdActivity`), the app starts in `MainActivity`, starts `SecondActivity`, starts `ThirdActivity`, calls `finish()`, and then starts `SecondActivity`. Which activity is currently at the top of the stack, and how many total activity instances are on the stack?
10. When an activity is popped off the stack via `finish()`, which lifecycle methods run and in what exact order? Which of those methods does *not* run if the activity simply goes into the background?
11. Which lifecycle method is **not** called after `onCreate()` when an activity first starts, but **is** automatically called when the activity comes back to the top of the stack? Which two lifecycle methods are called in **both** situations?
12. Inside `MainActivity`, write the two lines of Kotlin code to create and display a **short** `Toast` that displays `"Saved!"`.
13. Assume `sp` is a valid `SharedPreferences` reference. Write the Kotlin code to (a) read an `Int` with key `"score"` (defaulting to `0` if not found), and (b) write the `Float` value `4.5f` with key `"gpa"` and persist it to storage.

Part B Practice: Moving a View & Multi-Activity Persistence

14. **Dragging a Label in `RelativeLayout` (`5-Touches` Version 1)**: Assume `MainActivity` implements `View.OnTouchListener` and declares instance variables `tv : TextView`, `params : RelativeLayout.LayoutParams`, and integers `startX`, `startY`, `startTouchX`, `startTouchY`.
    - (a) Inside `onCreate`, instantiate `params` with width `250` and height `150`, set its initial X position to `40` and Y position to `80`, add `tv` to a `RelativeLayout` named `rl` using `params`, and register the touch listener on `tv`.
    - (b) Complete the `onTouch(v: View?, event: MotionEvent?): Boolean` method so that touching and dragging `tv` moves it smoothly with the user's finger.
15. **Starting a Second Activity & Sharing a Model (`4-Mortgage`)**:
    - (a) Declare a static-like reference `game : TicTacToe` inside `MainActivity` so another activity `ScoreActivity` can access it as `MainActivity.game`.

- (b) Write the body of `fun showScores(v: View)` inside `MainActivity` to launch `ScoreActivity`.

Answer Key for Practice Exam Questions

1. `android:inputType="numberDecimal"`
2. `android:layout_toRightOf="@+id/label_bill"` (or `"@id/label_bill"`)
3. ```
   <style name="headerStyle">
       <item name="android:textSize">28sp</item>
   </style>
   ```
4. Method name: `onTouch`. Signature: `override fun onTouch( v : View?, event : MotionEvent? ) : Boolean` (takes `View?` and `MotionEvent?`, returns `Boolean`).
5. Returning `true` **consumes** the touch event (no further views receive it); returning `false` **propagates** the event to Views underneath the current view.
6. `event.rawX` returns the X coordinate within the **entire screen**, whereas `event.x` returns the X coordinate relative to the **View that originated the event**. They are equal when the listener is registered on the full-screen content view `findViewById(android.R.id.content)`.
7. **One** `ACTION_DOWN` event, followed by **many** (discrete) `ACTION_MOVE` events, followed by **one** `ACTION_UP` event.
8. `var topView : View = findViewById( android.R.id.content )`
9. Stack trace: `[Main]` → `[Main, Second]` → `[Main, Second, Third]` → `finish()` pops `Third` leaving `[Main, Second]` → starting `SecondActivity` pushes a new instance: `[Main, Second, Second]`. **`SecondActivity`** is at the top of the stack, and there are **3** activity instances on the stack.
10. Order when popped off stack: **`onPause` (first)**, **`onStop` (second)**, **`onDestroy` (third)**. When an activity merely goes to the background, **`onDestroy`** does NOT run.
11. Only on return (not after `onCreate`): **`onRestart`**. Called both after `onCreate` and on return: **`onStart`** and **`onResume`**.
12. ```
    var t : Toast = Toast.makeText( this, "Saved!", Toast.LENGTH_SHORT )
    t.show( )
    ```
13. ```
    // (a) Read Int:
    var score : Int = sp.getInt( "score", 0 )
    // (b) Write Float and commit:
    var editor : SharedPreferences.Editor = sp.edit( )
    editor.putFloat( "gpa", 4.5f )
    editor.commit( )
    ```
    ```

14.
```
// (a) Inside onCreate:
params = RelativeLayout.LayoutParams( 250, 150 )
params.leftMargin = 40
params.topMargin = 80
rl.addView( tv, params )
tv.setOnTouchListener( this )

// (b) Inside onTouch:
override fun onTouch( v : View?, event : MotionEvent? ) : Boolean {
    if ( event != null ) {
        var action : Int = event.action
        if ( action == MotionEvent.ACTION_DOWN ) {
            startX = params.leftMargin
            startY = params.topMargin
            startTouchX = event.rawX.toInt( )
            startTouchY = event.rawY.toInt( )
        } else if ( action == MotionEvent.ACTION_MOVE ) {
            params.leftMargin = startX + ( event.rawX.toInt( ) - startTouchX )
            params.topMargin = startY + ( event.rawY.toInt( ) - startTouchY )
            tv.layoutParams = params
        }
    }
    return true
}
```

15.
```
// (a) Inside MainActivity class body:
companion object {
    lateinit var game : TicTacToe
}

// (b) Launching ScoreActivity:
fun showScores( v : View ) {
    var myIntent : Intent = Intent( this, ScoreActivity::class.java )
    this.startActivity( myIntent )
}
```